

МДК.01.04 Системное программирование

Задачи по функциям

Звёздный прямоугольник 1

Напишите функцию `draw_box()`, которая выводит звёздный прямоугольник с размерами 14×10 в соответствии с образцом:

```
*****  
*      *  
*      *  
*      *  
*      *  
*      *  
*      *  
*      *  
*      *  
*      *  
*      *  
*      *  
*      *  
*      *  
*****
```

Примечание. Для вывода прямоугольника используйте цикл `for`.

Звёздный треугольник 1

Напишите функцию `draw_triangle()`, которая выводит звёздный прямоугольный треугольник с катетами, равными 10, в соответствии с образцом:

```
*  
**  
***  
****  
*****  
*****  
*****  
*****  
*****  
*****  
*****
```

Примечание. Для вывода треугольника используйте цикл `for`.

Звёздный треугольник

Напишите функцию `draw_triangle(fill, base)`, которая принимает два параметра:

- `fill` – символ заполнитель;
- `base` – величина основания равнобедренного треугольника;

а затем выводит его.

Примечание. Гарантируется, что основание треугольника – нечётное число.

► Тестовые данные 

Sample Input 1:

```
*
9
```

Sample Output 1:

```
*
**
***
****
*****
****
***
**
*
```

Sample Input 2:

```
+
5
```

Sample Output 2:

```
+
++
+++
++
+
```

ФИО

Напишите функцию `print_fio(name, surname, patronymic)`, которая принимает три параметра:

- `name` – имя человека;
- `surname` – фамилия человека;
- `patronymic` – отчество человека;

а затем выводит на печать ФИО человека.

Примечание. Предусмотрите тот факт, что все три буквы в ФИО должны иметь верхний регистр.

► Тестовые данные 

Sample Input 1:

Александр
Пушкин
Сергеевич

Sample Output 1:

ПАС

Сумма цифр

Напишите функцию `print_digit_sum()`, которая принимает одно натуральное число `num` и выводит на печать сумму его цифр.

► Тестовые данные 

Sample Input 1:

12345

Sample Output 1:

15

Конвертер километров

Напишите функцию `convert_to_miles(km)`, которая принимает в качестве аргумента расстояние в километрах и возвращает расстояние в милях. Формула для преобразования: `мили = километры * 0.6214`.

Примечание. Приведённый ниже код:

```
print(convert_to_miles(1))
print(convert_to_miles(5))
print(convert_to_miles(10))
```

должен выводить:

```
0.6214
3.107
6.214
```

► Тестовые данные 

Sample Input 1:

1

Sample Output 1:

0.6214

Количество дней

Напишите функцию `get_days(month)`, которая принимает в качестве аргумента номер месяца и возвращает количество дней в данном месяце.

Примечание 1. Гарантируется, что передаваемый аргумент находится в диапазоне от 1 до 12 (включительно).

Примечание 2. Считайте, что год является невисокосным.

Примечание 3. Приведённый ниже код:

```
print(get_days(1))
print(get_days(2))
print(get_days(9))
```

должен выводить:

```
31
28
30
```

► Тестовые данные 

Sample Input 1:

1

Sample Output 1:

31

Делители 1

Напишите функцию `get_factors(num)`, принимающую в качестве аргумента натуральное число и возвращающую список всех делителей данного числа.

Примечание. Приведённый ниже код:

```
print(get_factors(1))
print(get_factors(5))
print(get_factors(10))
```

должен выводить:

```
[1]
[1, 5]
[1, 2, 5, 10]
```

► Тестовые данные 

Sample Input 1:

1

Sample Output 1:

[1]

Sample Input 2:

5

Sample Output 2:

[1, 5]

Делители 2

Напишите функцию `number_of_factors(num)`, принимающую в качестве аргумента число и возвращающую количество делителей данного числа.

Примечание 1. Используйте уже реализованную функцию `get_factors(num)` из предыдущей задачи.

Примечание 2. Приведённый ниже код:

```
print(number_of_factors(1))
print(number_of_factors(5))
print(number_of_factors(10))
```

должен выводить:

```
1
2
4
```

► Тестовые данные 

Sample Input 1:

1

Sample Output 1:

1

Sample Input 2:

5

Sample Output 2:

2

Найти всех

Напомним, что строковый метод `find('a')` возвращает местоположение первого вхождения символа `a` в строке. Проблема заключается в том, что данный метод не находит местоположение всех символов `a`.

Напишите функцию с именем `find_all(target, symbol)`, которая принимает два аргумента: строку `target` и символ `symbol` и возвращает список, содержащий все местоположения этого символа в строке.

Примечание 1. Если указанный символ не встречается в строке, то следует вернуть пустой список.

Примечание 2. Приведённый ниже код:

```
print(find_all('abcdabcaaa', 'a'))  
print(find_all('abcadbcaaa', 'e'))  
print(find_all('abcadbcaaa', 'd'))
```

должен выводить:

```
[0, 4, 7, 8, 9]  
[]  
[4]
```

► Тестовые данные 

Sample Input 1:

```
abcdabcaaa  
a
```

Sample Output 1:

```
[0, 4, 7, 8, 9]
```

Merge lists 1

Напишите функцию `merge(list1, list2)`, которая принимает в качестве аргументов два отсортированных по возрастанию списка, состоящих из целых чисел, и объединяет их в один отсортированный список.

Примечание 1. Списки `list1` и `list2` могут иметь разную длину.

Примечание 2. Можно использовать списочный метод `sort()`, а можно обойтись и без него. 😊

Примечание 3. Приведённый ниже код:

```
print(merge([1, 2, 3], [5, 6, 7, 8]))
print(merge([1, 7, 10, 16], [5, 6, 13, 20]))
```

должен выводить:

```
[1, 2, 3, 5, 6, 7, 8]
[1, 5, 6, 7, 10, 13, 16, 20]
```

► Тестовые данные 

Sample Input 1:

```
1 2 3
5 6 7 8
```

Sample Output 1:

```
[1, 2, 3, 5, 6, 7, 8]
```

Merge lists 2

На вход программе подаются число n , а затем n строк, содержащих целые числа в порядке возрастания. Из данных строк формируются списки чисел. Напишите программу, которая объединяет указанные списки в один отсортированный список с помощью функции `quick_merge()`, а затем выводит его.

Формат входных данных

На вход программе подаются натуральное число n , а затем n строк, содержащих целые числа в порядке возрастания, разделённые символом пробела.

Формат выходных данных

Программа должна вывести элементы окончательного отсортированного списка каждое через пробел.

► Тестовые данные 

Sample Input 1:

```
3
1 2 3 4
5 6 7
10 11 17
```

Sample Output 1:

```
1 2 3 4 5 6 7 10 11 17
```

Is the Number Prime? 🌶️

Напишите функцию `is_prime(num)`, которая принимает в качестве аргумента натуральное число и возвращает значение `True`, если число является простым, или `False` в противном случае.

Примечание. Приведённый ниже код:

```
print(is_prime(1))
print(is_prime(10))
print(is_prime(17))
```

должен выводить:

```
False
False
True
```

► Тестовые данные 

Sample Input 1:

```
1
```

Sample Output 1:

```
False
```

Sample Input 2:

```
10
```

Sample Output 2:

```
False
```

Next Prime 🌶️🌶️

Напишите функцию `get_next_prime(num)`, которая принимает в качестве аргумента натуральное число `num` и возвращает первое простое число, большее числа `num`.

Примечание 1. Используйте функцию `is_prime()` из предыдущей задачи.

Примечание 2. Приведённый ниже код:

```
print(get_next_prime(6))
print(get_next_prime(7))
print(get_next_prime(14))
```

должен выводить:

```
7
11
17
```

► Тестовые данные 🟢

Sample Input 1:

6

Sample Output 1:

7

Good password 🌶️

Напишите функцию `is_password_good(password)`, которая принимает в качестве аргумента строковое значение пароля `password` и возвращает значение `True`, если пароль является надёжным, или `False` в противном случае.

Пароль является надёжным, если:

- его длина не менее 8 символов;
- он содержит как минимум одну заглавную букву (верхний регистр);
- он содержит как минимум одну строчную букву (нижний регистр);
- он содержит хотя бы одну цифру.

Примечание. Приведённый ниже код:

```
print(is_password_good('aabbCC110P'))
print(is_password_good('abC1pu'))
```

должен выводить:

```
True
False
```

► Тестовые данные 🟢

Sample Input 1:

aaAA12qqp

Sample Output 1:

True

Ровно в одном 1

Напишите функцию `is_one_away(word1, word2)`, которая принимает в качестве аргументов два слова `word1` и `word2`. Функция должна возвращать значение `True`, если слова имеют одинаковую длину и отличаются одним символом на одной и той же позиции, или `False` в противном случае.

Примечание. Приведённый ниже код:

```
print(is_one_away('bike', 'hike'))
print(is_one_away('water', 'wafer'))
print(is_one_away('abcd', 'abpo'))
print(is_one_away('abcd', 'abcde'))
```

должен выводить:

```
True
True
False
False
```

► Тестовые данные 

Sample Input 1:

```
bike
hike
```

Sample Output 1:

```
True
```

Палиндром 🌶️

Напишите функцию `is_palindrome(text)`, которая принимает в качестве аргумента строку `text` и возвращает значение `True`, если указанный текст является палиндромом, или `False` в противном случае.

Примечание 1. Палиндром – это строка, которая читается одинаково в обоих направлениях

Примечание 2. При проверке считайте большие и маленькие буквы одинаковыми, а также игнорируйте пробелы и символы `,.!?-.`

Примечание 3. Приведённый ниже код:

```
print(is_palindrome('A роза упала на лапу Азора.'))
print(is_palindrome('Gabler Ruby - burrel bag!'))
print(is_palindrome('BEEGEEK'))
```

должен выводить:

```
True
True
False
```

► Тестовые данные 

Sample Input 1:

```
Standart - smallest, sell Amstrad nats.
```

Sample Output 1:

```
True
```

BEEGEEK 🐝 🌶️

BEEGEEK наконец-то открыл свой банк, в котором используются специальные банкоматы с необычным паролем.

Действительный пароль BEEGEEK банка имеет вид `a:b:c`, где `a`, `b` и `c` – натуральные числа. Поскольку основатель **BEEGEEK** фанатеет от математики, то он решил:

- число `a` – должно быть палиндромом;
- число `b` – должно быть простым;
- число `c` – должно быть чётным.

Напишите функцию `is_valid_password(password)`, которая принимает в качестве аргумента строковое значение пароля `password` и возвращает значение `True`, если пароль является действительным паролем BEEGEEK банка, или `False` в противном случае.

Примечание. Приведённый ниже код:

```
print(is_valid_password('1221:101:22'))
print(is_valid_password('565:30:50'))
print(is_valid_password('112:7:9'))
print(is_valid_password('1221:101:22:22'))
```

должен выводить:

```
True
False
False
False
```

Правильная скобочная последовательность 🌶️

Напишите функцию `is_correct_bracket(text)`, которая принимает в качестве аргумента непустую строку `text`, состоящую из символов `(` и `)` и возвращает значение `True`, если поступившая на вход строка является правильной скобочной последовательностью, или `False` в противном случае.

Примечание 1. Правильной скобочной последовательностью называется строка, состоящая только из символов `(` и `)`, где каждой открывающей скобке найдётся парная закрывающая скобка (при этом каждая открывающая скобка должна быть левее соответствующей ей закрывающей скобки).

Примечание 2. Приведённый ниже код:

```
print(is_correct_bracket('()()()()'))  
print(is_correct_bracket(')()()('))
```

должен выводить:

```
True  
False
```

► Тестовые данные 🟢

Sample Input 1:

`((()))`

Sample Output 1:

`True`

Sample Input 2:

`()()`

Sample Output 2:

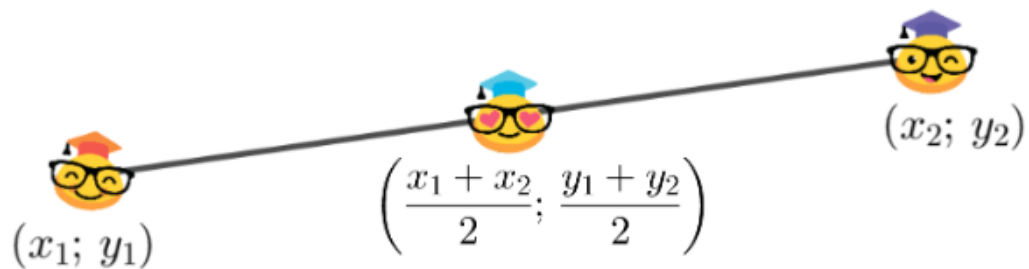
`True`

Середина отрезка

Напишите функцию `get_middle_point(x1, y1, x2, y2)`, которая принимает в качестве аргументов координаты концов отрезка $(x_1; y_1)$ и $(x_2; y_2)$ и возвращает координаты точки являющейся серединой данного отрезка.

Примечание 1. Координаты середины отрезка с концами в точках $(x_1; y_1)$ и $(x_2; y_2)$ вычисляются по формуле:

$$\left(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2} \right).$$



Примечание 2. Приведённый ниже код:

```
print(get_middle_point(0, 0, 10, 0))
print(get_middle_point(1, 5, 8, 3))
```

должен выводить:

```
5.0 0.0
4.5 4.0
```

► Тестовые данные 

Sample Input 1:

```
1
1
2
2
```

Sample Output 1:

```
1.5 1.5
```

Sample Input 2:

```
-10
10
0
100
```

Sample Output 2:

```
-5.0 55.0
```

Площадь и длина

Напишите функцию `get_circle(radius)`, которая принимает в качестве аргумента радиус окружности и возвращает два значения: длину окружности и площадь круга, ограниченного данной окружностью.

Примечание 1. Длина окружности и площадь круга радиуса r вычисляются по формулам:

$$C = 2\pi r, S = \pi r^2$$

Примечание 2. Для числа π используйте глобальную константу из модуля `math`.

Примечание 3. Приведённый ниже код:

```
print(get_circle(1))
print(get_circle(1.5))
```

должен выводить:

```
6.283185307179586 3.141592653589793
9.42477796076938 7.0685834705770345
```

► Тестовые данные 

Sample Input 1:

1

Sample Output 1:

6.283185307179586 3.141592653589793

Sample Input 2:

7.2

Sample Output 2:

45.23893421169302 162.8601631620949

Корни уравнения 🌶️🌶️

Напишите функцию `solve(a, b, c)`, которая принимает в качестве аргументов три целых числа `a`, `b`, `c` – коэффициенты квадратного уравнения $ax^2 + bx + c = 0$ и возвращает его корни в порядке возрастания.

Примечание 1. С подобной [задачей](#) мы уже сталкивались.

Примечание 2. Гарантируется, что квадратное уравнение имеет хотя бы один корень.

Примечание 3. Приведённый ниже код:

```
print(solve(1, -4, -5))
print(solve(-2, 7, -5))
print(solve(1, 2, 1))
```

должен выводить:

```
-1.0 5.0
1.0 2.5
-1.0 -1.0
```

Примечание 4. Если уравнение имеет только один корень, нужно вернуть два числа, равных этому корню.

► Тестовые данные 

Sample Input 1:

```
1
-4
-5
```

Sample Output 1:

```
-1.0 5.0
```